Listing 7.14 `oai_agent.py` (`get_response_stream`: execute tool calls)

```
if tool_calls:     #1
    available_functions = {
        action["name"]: action["pointer"] for action in self.actions
    }     #2
    self.messages.append(
        response_message
    )
    for tool_call in tool_calls:     #3
        function_name = tool_call.function.name
        function_to_call = available_functions[function_name]
        function_args = json.loads(tool_call.function.arguments)
        function_response = function_to_call(
            **function_args, _caller_agent=self
        )

        self.messages.append(
            {
                "tool_call_id": tool_call.id,
                "role": "tool",
                "name": function_name,
                "content": str(function_response),
            }
        )
    second_response = self.client.chat.completions.create(
        model=self.model,
        messages=self.messages,
    )      #4
    response_message = second_response.choices[0].message
```

#1 Proceeds if tool calls are detected in the LLM response
#2 Loads pointers to the actual function implementations for code execution
#3 Loops through all the calls the LLM wants to call; there can be several.
#4 Performs a second LLM call with the results of the tool calls

To demo this, start up Nexus in the debugger by pressing F5. Then, select the two test actions—`recommend` and `get_current_weather`—and the terse persona/profile Olly. Figure 7.9 shows the result of entering a query and the agent responding by using both tools in its response.